

Chapter 1. Exam Details and Resources

This chapter addresses the most frequently asked questions by candidates preparing to successfully pass the [Certified Kubernetes Administrator \(CKA\) exam](#). Later chapters will give you a summary of [Kubernetes' benefits and architecture](#) and how to [interact with a Kubernetes cluster using kubectl](#).

Kubernetes Certification Learning Path

The Cloud Native Computing Foundation (CNCF) offers five different Kubernetes certifications. [Figure 1-1](#) categorizes each of them by target audience.

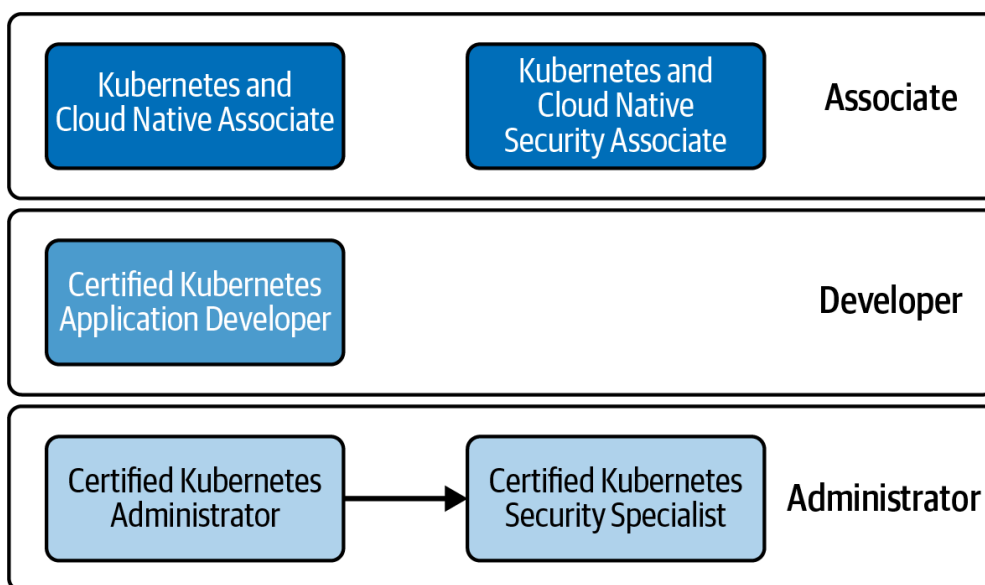


Figure 1-1. Kubernetes certifications learning path

The target audience for associate-level certifications is beginners to the cloud and Kubernetes. Associate-level certification exams use a multiple-choice format. You will not have to interact with a Kubernetes cluster in an interactive environment.

Practitioner-level certifications are meant for developers and administrators with preexisting Kubernetes experience. Exams in this category require you to solve problems in multiple Kubernetes environments hands-on. You will find that the CKA is geared to administrators of a Kubernetes cluster and does not require any other certification as a prerequisite.

If you pass all Kubernetes certifications, you can call yourself a [Kubestronaut](#). The Kubestronaut program recognizes community members that are able to demonstrate their skills in the most important core areas of Kubernetes.

Let's have a brief look at each certification to see if the CKA is the right fit for you.

Kubernetes and Cloud Native Associate (KCNA)

[KCNA](#) is an entry-level certification program for anyone interested in cloud native application development, runtime environments, and tooling. While the exam does cover Kubernetes, it does *not* expect you to interact with a cluster directly. This exam consists of multiple-choice questions and is suitable to candidates interested in the topic with a broad exposure to the ecosystem.

Kubernetes and Cloud Native Security Associate (KCSA)

The [KCSA](#) verifies your basic knowledge of security concepts and their application in a Kubernetes cluster. The breadth, depth, and format of the program is definitely more advanced than the KCNA. I personally would not categorize this exam as suitable to beginners to security concepts. You may want to take the KCSA after gaining more hands-on and in-depth exposure to Kubernetes, as the exam questions can be quite hard to answer.

Certified Kubernetes Application Developer (CKAD)

The [CKAD](#) exam focuses on verifying your ability to build, configure, and deploy a microservices-based application to Kubernetes. You are not expected to actually implement an application; however, the exam is suitable for developers familiar with topics like application architecture, runtimes, and programming languages.

Certified Kubernetes Administrator (CKA)

The target audience for the [CKA](#) exam is DevOps practitioners, system administrators, and site reliability engineers. This exam tests your ability to perform in the role of a Kubernetes administrator, which includes tasks like cluster, network, storage, and beginner-level security management, with emphasis on troubleshooting scenarios.

Certified Kubernetes Security Specialist (CKS)

The [CKS](#) exam expands on the topics verified by the CKA exam. Passing the CKA is a prerequisite before you can sign up for the CKS exam. For this certification, you are expected to have a deeper knowledge of Kubernetes security. The curriculum covers topics like applying best practices for building containerized applications and ensuring a secure Kubernetes runtime environment.

Exam Objectives

Kubernetes clusters need to be installed, configured, and maintained by skilled professionals. That's the job of a Kubernetes administrator. The CKA certification program verifies a deep understanding of the typical administration tasks encountered on the job: more specifically, Kubernetes cluster maintenance, networking, storage solutions, and troubleshooting applications and cluster nodes.

This book focuses on getting you ready for the CKA exam. I will give a little bit of background on why Kubernetes is important to administrators before dissecting the topics important to the exam.

KUBERNETES VERSION USED DURING THE EXAM

At the time of writing, the exam is based on Kubernetes 1.33. All content in this book will follow the features, APIs, and command-line support for that version. It's possible that future versions will break backward compatibility. While preparing for the certification, review the [Kubernetes release notes](#) and practice with the Kubernetes version used during the exam to avoid unpleasant surprises. The exam environment will be aligned with the most recent Kubernetes minor version within approximately four to eight weeks of the Kubernetes release date.

Curriculum

The following overview lists the high-level sections, or domains, of the exam and their scoring weights:

- 25%: [Cluster Architecture, Installation, and Configuration](#)
- 15%: [Workloads and Scheduling](#)
- 20%: [Servicing and Networking](#)
- 10%: [Storage](#)
- 30%: [Troubleshooting](#)

The next sections detail each domain.

Cluster Architecture, Installation, and Configuration

This section of the curriculum dives deep into all aspects of Kubernetes clusters. It covers the fundamental architecture of Kubernetes, including the distinction between the control plane and worker nodes, high-availability configurations, and the tools needed for installing, upgrading, and maintaining a cluster. Additionally, you'll explore key extension interfaces and learn practical skills such as installing a cluster from scratch, upgrading its version, and backing up/restoring the etcd database.

The CNCF has also incorporated related topics into this domain. For instance, mastering role-based access control (RBAC) is essential for administrators to effectively manage access to cluster resources. You'll also become proficient in installing Kubernetes operators and using tools like Kustomize and Helm to discover and deploy cluster components efficiently.

Workloads and Scheduling

Administrators must have a solid understanding of Kubernetes concepts essential for managing cloud native applications effectively. The domain focuses on these critical aspects. It covers key resources such as Deployments, ReplicaSets, and configuration management using ConfigMaps and Secrets.

When a new Pod is created, the Kubernetes scheduler assigns it to an available node based on predefined criteria. Scheduling rules, such as node affinity and taints/tolerations, help fine-tune this process to meet specific requirements. For exam preparation, it's crucial to grasp the various factors and concepts involved in the Kubernetes scheduling algorithm to ensure optimal workload placement and performance.

Servicing and Networking

A cloud native microservice rarely operates in isolation. More often than not, it interacts with other microservices or external systems. For administrators, understanding Pod-to-Pod communication, exposing applications to external clients, and configuring cluster networking is crucial for maintaining a fully functional system.

This domain of the exam evaluates your knowledge of essential Kubernetes networking primitives, including Services, Ingress,

NetworkPolicy, and the Gateway API. Mastering these components ensures you can effectively manage and secure communication within and outside the cluster.

Storage

This domain focuses on the various types of volumes used for reading and writing data in Kubernetes. As an administrator, you must understand how to create, configure, and manage these volumes effectively.

PersistentVolumes (PVs) play a crucial role in ensuring data persistence, even after a cluster node restarts. You'll need to demonstrate the ability to mount a PersistentVolume to a specific path within a container and understand the underlying mechanics. Additionally, it's essential to grasp the differences between static and dynamic provisioning to manage storage resources efficiently.

Troubleshooting

In production Kubernetes clusters, issues are bound to arise. Applications may misbehave, become unresponsive, or even become completely inaccessible. Additionally, cluster nodes might crash or face configuration problems. Developing effective troubleshooting strategies is critical to quickly identifying and resolving these situations to minimize downtime and disruption.

This domain carries the highest scoring weight in the exam. You will encounter realistic scenarios that require you to diagnose problems and implement the appropriate solutions. Mastering these skills ensures you can maintain the stability and reliability of Kubernetes environments under pressure.

Involved Kubernetes Primitives

Some of the exam objectives can be covered by understanding the relevant core Kubernetes primitives. Be aware that the exam combines multiple concepts in a single problem. Refer to [Figure 1-2](#) as a guide to the applicable Kubernetes resources and their relationships.

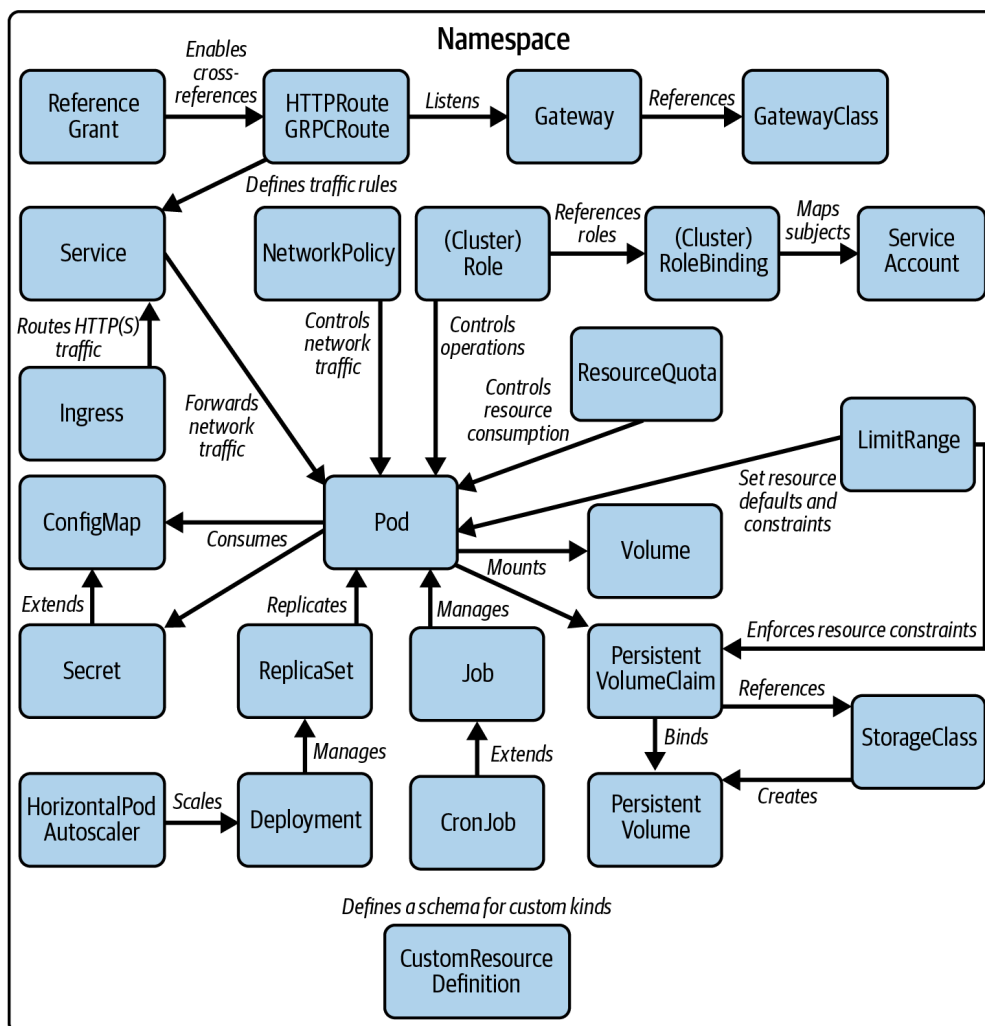


Figure 1-2. Kubernetes primitives relevant to the exam

Documentation

During the exam, you are permitted to open a well-defined list of web pages as a reference. You can freely browse those pages and copy-paste code to the exam terminal. The official Kubernetes documentation includes the reference manual and the blog. In addition, you can also browse the Helm documentation:

- Reference manual: <https://kubernetes.io/docs>
- Blog: <https://kubernetes.io/blog>
- Helm: <https://helm.sh/docs>

Having the Kubernetes documentation pages at hand is extremely valuable, but make sure you know *where* to find the relevant information within those pages. In preparation for the test, read all the documentation pages from start to end at least once. Don't forget the search functionality of the official documentation pages. For reference, [Appendix B](#) maps the exam objectives to the book chapters covering the topics and the relevant Kubernetes documentation pages.

Using a search term will likely lead you to the right documentation pages quicker than navigating the menu items. Copying and pasting code snippets from the documentation into the console of the exam environment works reasonably well. You may have to adjust the YAML indentation manually as the proper formatting can get lost in the process.

Exam Environment and Tips

To take the exam, you must purchase a registration voucher, which can be acquired on the [CNCF training and certification web page](#). On occasion, the CNCF offers discounts for the voucher (e.g., around the US Thanksgiving holiday). Those discount offers are often announced on the [Linux Foundation LinkedIn page](#).

After you purchase the voucher, you can schedule a time for the exam with [PSI](#), the company conducting the test virtually. In-person exams at a testing facility are not available. On the day of your scheduled test, you'll be asked to log into the test platform with a URL provided to you by email. You'll be asked to enable the audio and video feed on your computer to discourage cheating. A proctor will oversee your actions via audio/video feed and terminate the session if they think you are not following the rules.

EXAM ATTEMPTS

The voucher you purchased grants two attempts to pass the exam. I recommend preparing reasonably well before taking the test on the first attempt. It will give you a fair chance to pass the test and provide a good impression of the exam environment and the complexity of the questions. Don't sweat it if you do not pass the test on the first attempt. You've got another free shot.

The exam has a time limit of two hours. During that time, you'll need to solve hands-on problems on a real, predefined Kubernetes cluster. Every question will state the cluster you need to work on. This practical approach to gauge a candidate's skill set is superior to tests with multiple-choice questions, as you can translate the knowledge directly to tasks performed on the job.

I highly recommend reading the [FAQ for the exam](#). You will find answers to most of your pressing questions there, including system requirements for your machine, scoring, certification renewal, and retake requirements.

Candidate Skills

The certification assumes that you have a basic understanding of Kubernetes. You should be familiar with Kubernetes internals, its core concepts, and the command-line tool `kubectl`. The CNCF offers a free [“Introduction to Kubernetes” course](#) for beginners to Kubernetes.

Here’s a brief overview of the background knowledge you need to increase your likelihood of passing the exam:

Kubernetes architecture and concepts

The exam won’t ask you to install a Kubernetes cluster from scratch. Read up on the basics of Kubernetes and its architectural components. Reference [Chapter 2](#) for a jump start on Kubernetes architecture and concepts.

The `kubectl` CLI tool

The `kubectl` command-line tool is the central tool you will use during the exam to interact with the Kubernetes cluster. Even if you have only a little time to prepare for the exam, it’s essential to practice operating `kubectl`, as well as its commands and their relevant options. You will have no access to the [web dashboard UI](#) during the exam. [Chapter 3](#) provides a short summary of the most important ways of interacting with a Kubernetes cluster.

Kubernetes cluster maintenance tools

Installing a Kubernetes cluster from scratch and upgrading the Kubernetes version of an existing cluster is performed using the tool `kubeadm`. It’s important to understand its usage and the relevant steps to walk through the process. Reference [Chapter 4](#) for more information. Additionally, you need to have a good understanding of the tools `etcdctl` and `etcdutl`, including their command-line options for backing up and restoring the etcd database covered in [Chapter 5](#).

Working knowledge of a container runtime engine

Kubernetes uses a container runtime engine for managing images. The default container runtime engine in Kubernetes is containerd. At a minimum, understand the difference between container images and containers, and their purpose. This topic goes beyond the coverage of this book and isn’t directly tested for in the exam.

Foundational Linux knowledge

The exam is conducted entirely through a Linux terminal, requiring solid command-line skills. You should be comfortable navigating the file system, managing files and directories, and understanding file permissions. Familiarity with text editors like `vi` or `vim` is essential, as you'll need to edit YAML files frequently. Basic process management skills such as viewing running processes, checking resource usage, and understanding system logs are important for troubleshooting tasks. You should also be comfortable with common Linux commands for text processing (`grep` , `awk` , `sed`), file operations (`cat` , `less` , `head` , `tail`), and basic shell scripting concepts including piping, redirection, and environment variables. Knowledge of SSH for accessing remote nodes and basic networking commands (`netstat` , `curl` , `wget`) will also prove valuable during the exam.

Time Management

The Linux Foundation expects Kubernetes practitioners to be able to apply their knowledge to real-world scenarios by finding solutions to problems in a timely fashion. The exam typically contains 15–20 performance-based questions to be completed in 2 hours, giving you roughly 6–8 minutes per question on average.

Since no single strategy works for everyone, consider these approaches for managing the exam's time constraints. One option is to read through all questions first before attempting any, allowing you to mentally sort them by perceived difficulty and tackle easier ones first to build confidence and secure quick points. Alternatively, you could take questions in the order presented but strictly time-box each to 4–5 minutes (leaving buffer time for review), marking incomplete ones for return, then using your second pass to revisit flagged questions ordered by difficulty or remaining work needed. A hybrid approach combines both strategies: initially scan all questions to assess difficulty, then work through them with strict time-boxes, leaving a third pass for reassessing and completing questions where you've made partial progress or gained clarity from working through similar problems.

The key is having a deliberate strategy rather than getting stuck on difficult questions early, as the exam rewards breadth of knowledge and efficient time management over perfecting individual complex scenarios. Remember that partial credit is often available, so it's better to attempt all questions than to perfect just a few.

Command-Line Tips and Tricks

Given that the command line is your solitary interface to the Kubernetes cluster, it's essential that you become extremely familiar with the `kubectl` tool and its available options. This section provides tips and tricks for making their use more efficient and productive.

Setting a Context and Namespace

The exam environment comes with multiple Kubernetes clusters already set up for you. Take a look at the [instructions](#) for a high-level, technical overview of those clusters. Each of the exam tasks need to be solved on a designated cluster, as outlined in its description. Furthermore, the instructions will ask you to work in a namespace other than `default`. Make sure to set the context and namespace as the first course of action before working on a question. The following command sets the context and the namespace as a one-time action:

```
$ kubectl config set-context <context-of-question> \
  --namespace=<namespace-of-question>
$ kubectl config use-context <context-of-question>
```

You can find a more detailed discussion of the context concept and the corresponding `kubectl` commands in [“Authentication with kubectl”](#).

For specific tasks, e.g., the upgrade process of a cluster node, you will need to open an interactive shell to the host machine of a cluster node. Use the `ssh <nodename>` command to achieve that.

Using the Alias for kubectl

In the course of the exam, you will have to execute the `kubectl` command tens or even hundreds of times. You might be an extremely fast typist; however, there's no point in fully spelling out the executable over and over again. The exam environment already sets up the alias `k` for the `kubectl` command.

In preparation for the exam, you can set up the same behavior on your machine. The following `alias` command maps the letter `k` to the full `kubectl` command:

```
$ alias k=kubectl
$ k version
```

Using kubectl Command Auto-Completion

Memorizing `kubectl` commands and command-line options takes a lot of practice. The exam environment comes with auto-completion enabled by default. You can find instructions for setting up auto-completion for the shell on your machine in the [Kubernetes documentation](#).

Internalizing Resource Short Names

Many of the `kubectl` commands can be quite lengthy. For example, the command for managing PersistentVolumeClaims is `persistentvolumeclaims`. Spelling out the full command can be error-prone and time-consuming. Thankfully, some of the longer commands come with a short-form usage. The command `api-resources` lists all available commands plus their short names:

```
$ kubectl api-resources
```

NAME	SHORTNAMES	APIGROUP	NAMESPACED	KIND
...				
persistentvolumeclaims	pvc		true	PersistentVolumeClaim
...				

Using `pvc` instead of `persistentvolumeclaims` results in a more concise and expressive command execution, as shown here:

```
$ kubectl describe pvc my-claim
```

Practicing and Practice Exams

Hands-on practice is extremely important when it comes to passing the exam. For that purpose, you'll need a functioning Kubernetes cluster environment. The following options stand out:

- I found it useful to run one or many virtual machines using [Vagrant](#) and [VirtualBox](#). Those tools help with creating an isolated Kubernetes environment that is easy to bootstrap and dispose on demand.
- It is relatively easy to install a simple Kubernetes cluster on your developer machine. The Kubernetes documentation provides various [installation options](#), depending on your operating system. [minikube](#) is useful when it comes to experimenting with more advanced features like Ingress or storage classes, as it provides the necessary functionality as add-ons that can be installed with a single command.

Alternatively, you can also give [kind](#) a try; it's another tool for running local Kubernetes clusters.

- If you're a subscriber to the [O'Reilly Learning Platform](#), you have unlimited access to scenarios running a [Kubernetes sandbox environment](#). In addition, you can test your knowledge with the help of the CKA exam prep [interactive labs](#).
- [Killercoda](#) hosts interactive scenarios for the exam contributed by the Kubernetes community.

You may also want to try one of the following commercial learning and practice resources:

- [Killer Shell](#) is a simulator with sample exercises for all Kubernetes certifications. If you purchase a voucher for the exam, you will be allowed two free sessions.
- Other online training providers offer video courses for the exam, some of which include an integrated Kubernetes practice environment. I would like to mention [KodeKloud](#) and [Pluralsight](#). You'll need to purchase a subscription to access the content for each course individually.

Summary

The exam is a completely hands-on test that requires you to solve problems in multiple Kubernetes clusters. You're expected to understand, use, and configure the Kubernetes primitives relevant to administrators. The exam curriculum subdivides those focus areas and puts different weights on topics, which determines their contributions to the overall score. Even though focus areas are grouped meaningfully, the curriculum doesn't necessarily follow a natural learning path, so it's helpful to cross-reference chapters in the book in preparation for the exam.

In this chapter, we discussed the exam environment and how to navigate it. The key to acing the exam is intense practice of `kubectl` to solve real-world scenarios. The next two chapters in [Part I](#) will provide a jump start to Kubernetes.

All chapters that discuss domain details give you an opportunity to practice hands-on. You will find sample exercises at the end of each chapter.